

# Documentación Técnica y Arquitectónica

## App Móvil: Farmacias de Turno

Alumno

16 de julio de 2026

## Índice

|                                                                    |          |
|--------------------------------------------------------------------|----------|
| <b>1. Introducción</b>                                             | <b>2</b> |
| <b>2. Acuerdos Arquitectónicos (Fase 0)</b>                        | <b>2</b> |
| 2.1. Metodología de Trabajo . . . . .                              | 2        |
| 2.2. Estandarización de Código . . . . .                           | 2        |
| 2.3. Restricciones de Infraestructura y Servicios . . . . .        | 2        |
| <b>3. Diseño del Sistema</b>                                       | <b>3</b> |
| 3.1. Diagrama de Contexto (Nivel 1) . . . . .                      | 3        |
| 3.2. Diagrama de Contenedores (Nivel 2) . . . . .                  | 3        |
| <b>4. Estrategias de Optimización y Rendimiento (Fase 4.1)</b>     | <b>5</b> |
| 4.1. Problema de Rendimiento (Renderizado Masivo) . . . . .        | 5        |
| 4.2. Solución Arquitectónica (Filtro Síncrono en RAM) . . . . .    | 5        |
| 4.3. Estrategia de Caché y Persistencia (Trabajo Futuro) . . . . . | 6        |
| <b>5. Estructura de Datos y Modelos (Payloads)</b>                 | <b>6</b> |
| 5.1. API del MINSAL (Farmacias de Turno Nacional) . . . . .        | 6        |
| 5.2. API de la UTEM (Farmacia más cercana) . . . . .               | 7        |
| 5.2.1. Hallazgo Técnico y Decisión de UI (Fase 4) . . . . .        | 7        |

# 1. Introducción

Este documento tiene como objetivo registrar de manera formal y evolutiva todas las decisiones arquitectónicas, diagramas y configuraciones de la aplicación móvil solicitada por el profesor Sebastián Salazar Molina.

La aplicación está desarrollada en **Flutter** (Dart), orientada principalmente al sistema operativo Android, e implementa integraciones con hardware (GPS), servicios en la nube (Firebase Google Sign-In) y APIs RESTful (para obtención de clima y farmacias de turno).

## 2. Acuerdos Arquitectónicos (Fase 0)

### 2.1. Metodología de Trabajo

Para evitar deuda técnica, el desarrollo se lleva a cabo mediante una metodología estrictamente iterativa. Cada componente debe ser diseñado, implementado mediante programación progresiva y finalmente validado mediante revisión de código antes de avanzar a la siguiente etapa.

### 2.2. Estandarización de Código

Se han definido las siguientes normas de calidad para el código fuente:

- **SDK Android:** `minSdkVersion 21` (Android 5.0) para asegurar compatibilidad con Firebase y Geolocator.
- **Application ID:** `ezekim.farmaciasgps` establecido como identificador único de Android.
- **Gestión de Estado:** Riverpod.
- **Arquitectura:** Feature-First (Agrupación por funcionalidades) y estricto apego a Principios SOLID (Responsabilidad Única) para evitar *God Functions*.
- **Nomenclatura:** `snake_case` para archivos, `PascalCase` para Clases/Enums, y `camelCase` para variables y funciones.
- **Documentación:** Código 100 % en idioma español con documentación activa para propósitos académicos.

### 2.3. Restricciones de Infraestructura y Servicios

Para garantizar la viabilidad académica del proyecto y evitar costos imprevistos, se ha establecido la directriz estricta de utilizar exclusivamente servicios en la nube y herramientas **100 % gratuitas (Cloud Free)**. Está terminantemente prohibido incorporar dependencias o APIs que requieran el ingreso de tarjetas de crédito o configuración de facturación. Un ejemplo de la aplicación de esta regla es el uso de OpenStreetMap en lugar del servicio comercial de Google Maps API.

### 3. Diseño del Sistema

#### 3.1. Diagrama de Contexto (Nivel 1)

El diagrama de contexto ilustra cómo el usuario interactúa con la aplicación móvil y los sistemas externos necesarios.

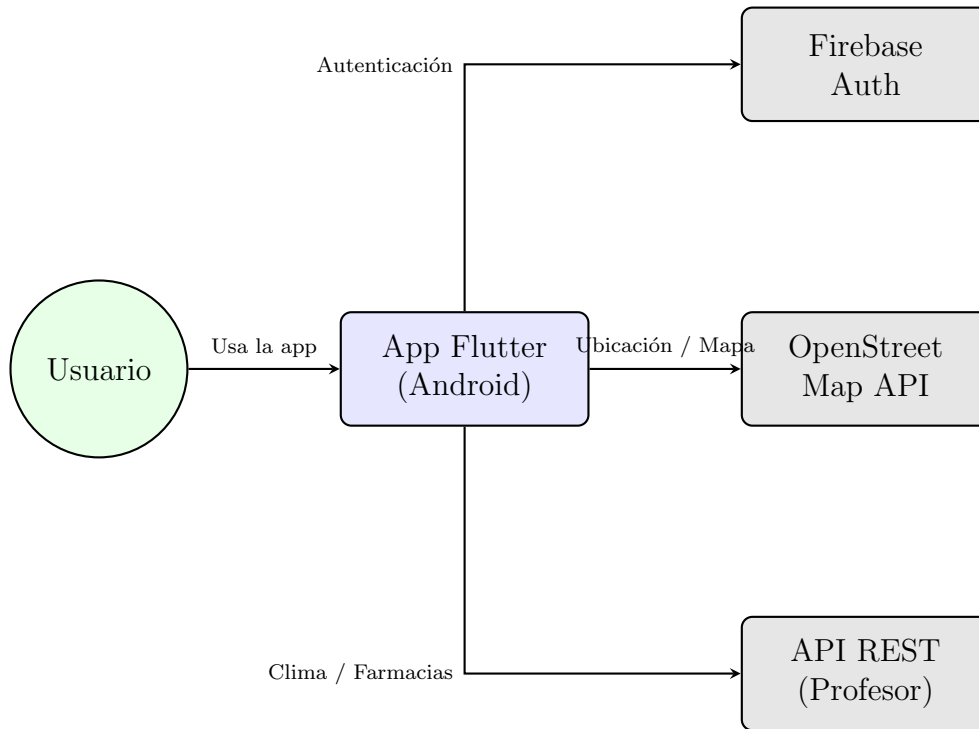


Figura 1: Diagrama de Contexto de la Aplicación.

#### 3.2. Diagrama de Contenedores (Nivel 2)

El diagrama de contenedores detalla la estructura interna de la aplicación Flutter (separación entre UI, Riverpod y Servicios).

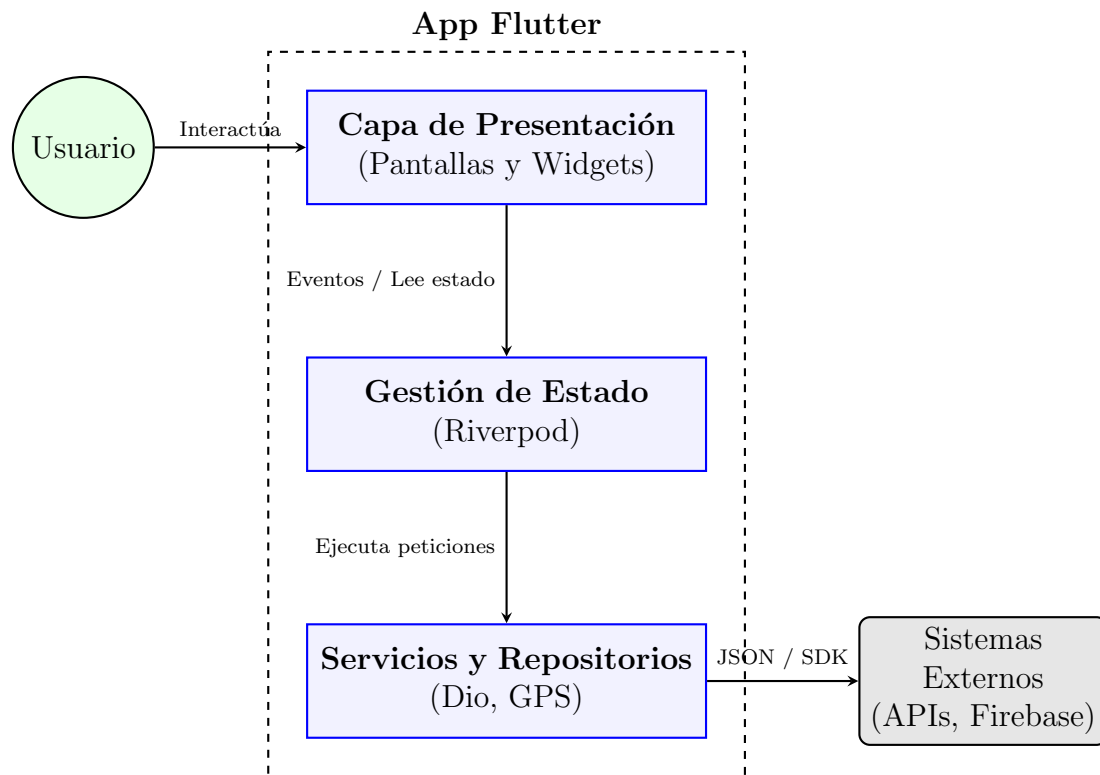


Figura 2: Diagrama de Contenedores Internos.

## Explicación del Flujo de Datos

Para comprender mejor la arquitectura de software planteada, es fundamental entender el rol de cada capa y cómo fluye la información desde el usuario hasta los servidores externos:

1. **Capa de Presentación (UI/UX):** Es la interfaz visual pura (botones, mapas, textos). Esta capa es "tonta", es decir, no contiene lógica de negocio ni hace peticiones a internet. Solo captura las acciones del usuario (ej. presionar "Buscar Farmacias") y muestra los datos que le entrega la capa inferior.
2. **Gestión de Estado (Riverpod):** Funciona como el cerebro.º puente intermedio. Cuando el usuario hace una acción en la UI, la Gestión de Estado se entera y decide qué hacer. Su trabajo principal es solicitar datos a los Servicios, mantener esos datos almacenados temporalmente en la memoria del teléfono, y avisarle a la UI de forma reactiva para que se actualice o redibuje cuando los datos cambien.
3. **Servicios y Repositorios:** Son los .ºbreros". Tienen el código específico para comunicarse con el mundo exterior. Utilizan herramientas como **Dio** para hacer peticiones HTTP a las APIs, o paquetes como **Geolocator** para obtener coordenadas del chip GPS.
4. **Sistemas Externos:** Son los proveedores finales de la información que consume la capa de Servicios. Incluyen:
  - **Firebase Auth:** Mantiene la persistencia de sesión segura usando las cuentas de Google del usuario.

- **OpenStreetMap API:** Brinda la cartografía de uso libre y gratuito para dibujar los mapas, evitando los costos de facturación de Google Maps.
- **API del Profesor:** Servidor RESTful que retorna la información climática y la lista de farmacias de turno en bruto (formato JSON).

### Ejemplo de un Flujo Completo (paso a paso):

1. El usuario abre la app y pulsa el botón "Ver farmacias cercanas.<sup>en</sup> la **UI**.
2. Ese botón no hace la búsqueda directa, sino que le avisa a **Riverpod** (Gestión de Estado) que se necesitan los datos.
3. Riverpod llama al **Servicio** de Farmacias y al Servicio de GPS.
4. Los Servicios hacen el "trabajo pesado": Encienden el GPS del teléfono y hacen la petición HTTP a la **API del profesor**.
5. La API responde con un texto en formato JSON. El Servicio toma ese texto, lo "traduce" lo convierte en una lista de objetos **Farmacia** que Dart entiende, devolviéndosela a Riverpod.
6. Riverpod guarda esa lista de farmacias en su memoria interna.
7. Automáticamente (y de forma reactiva), la **UI** detecta que Riverpod tiene nuevos datos, por lo que se redibuja.<sup>a</sup> sí misma mostrando finalmente los pines en el mapa para el usuario.

## 4. Estrategias de Optimización y Rendimiento (Fase 4.1)

Para la integración de los datos en tiempo real, se tomó la decisión arquitectónica de consumir de forma paralela dos fuentes de datos distintas: La API oficial del MINSAL (Farmacias de Turno a nivel nacional) y la API académica de la UTEM (Clima y Farmacia más cercana).

### 4.1. Problema de Rendimiento (Renderizado Masivo)

La API del MINSAL retorna la totalidad de las farmacias de turno del país (aproximadamente 2047 registros) en una única petición HTTP. Durante la fase de desarrollo, intentar renderizar los 2047 marcadores simultáneamente en `flutter_map` causaba una caída drástica en los cuadros por segundo (FPS) y provocaba bloqueos (hangs) en el hilo principal de la aplicación móvil.

### 4.2. Solución Arquitectónica (Filtro Síncrono en RAM)

Para mantener la máxima fluidez sin sacrificar la funcionalidad, se diseñó la siguiente estrategia mediante la gestión de estado de Riverpod:

1. **Descarga en Segundo Plano (Provider 1):** Al iniciar la aplicación, un proveedor aislado realiza la petición HTTP de 900KB al MINSAL. Esta operación se realiza una sola vez por ciclo de vida de la app y la lista de 2047 farmacias se aloja directamente en la memoria RAM.
2. **Filtro de Proximidad (Provider 2):** Un segundo proveedor reactivo escucha simultáneamente la lista en RAM y el flujo de eventos del GPS del dispositivo. Utilizando la librería `latlong2`, calcula la distancia geodésica (Haversine) entre el usuario y cada una de las 2047 farmacias.
3. **Renderizado Limitado:** El filtro descarta todas las farmacias que superen un radio de 5 kilómetros. Como resultado, la interfaz de usuario (UI) recibe de forma instantánea una lista reducida de 10 a 30 farmacias, garantizando que el mapa se renderice sin problemas de rendimiento.

### 4.3. Estrategia de Caché y Persistencia (Trabajo Futuro)

Dado que los turnos de las farmacias cambian diariamente (a las 08:00 AM o medianoche), la lista debe recargarse. Actualmente, si el usuario cierra y vuelve a abrir la aplicación 10 veces en un día, se realizarán 10 peticiones HTTP. Aunque el peso de 900KB no impacta significativamente en redes modernas, las buenas prácticas dictan evitar peticiones redundantes.

En una futura fase (Persistencia Local), se contempla la posibilidad de guardar esta lista en una base de datos local (como SQLite o SharedPreferences) y asignarle un Tiempo de Vida (TTL). La aplicación verificaría la hora local, y solo realizaría una nueva descarga si la caché tiene más de 24 horas de antigüedad o si se ha cruzado la hora de cambio de turno oficial.

## 5. Estructura de Datos y Modelos (Payloads)

Para optimizar el uso de memoria RAM, la aplicación no almacena todos los campos proporcionados por las APIs, sino que aplica un mapeo selectivo extrayendo únicamente los campos necesarios para renderizar el mapa y filtrar los horarios.

### 5.1. API del MINSAL (Farmacias de Turno Nacional)

El endpoint `/getLocalesTurnos.php` devuelve una lista de objetos JSON exclusivamente con las farmacias que están operativas en turno de emergencia o 24 horas. La cantidad de registros devueltos es dinámica y depende enteramente del momento del día en que se consulte (por ejemplo, en la madrugada la cantidad se reduce drásticamente respecto al día). Originalmente se utilizó erróneamente el endpoint `getLocales.php`, el cual retornaba todo el directorio nacional sin discriminar turnos.

```
{
  "fecha": "16-07-26",
  "local_id": "3",
  "local_nombre": "CRUZ VERDE",
  "comuna_nombre": "LIMACHE",
  "localidad_nombre": "LIMACHE",
```

```

"local_direccion": "URMENETA 99",
"funcionamiento_hora_apertura": "08:30:00",
"funcionamiento_hora_cierre": "18:30:00",
"local_telefono": "+56332415940",
"local_lat": "-32.9849921792696",
"local_lng": "-71.2757177058683",
"funcionamiento_dia": "jueves",
"fk_region": "6",
"fk_comuna": "59",
"fk_localidad": "17"
}

```

**Campos rescatados:** Se rescata `local_nombre`, `local_direccion`, `local_lat`, `local_lng`, `funcionamiento_hora_apertura`, `funcionamiento_hora_cierre` y `funcionamiento_dia`. Los datos de región o comuna son descartados ya que el filtro es puramente geodésico (coordenadas).

## 5.2. API de la UTEM (Farmacia más cercana)

El endpoint `/v1/farmacias/{lat}/{lng}` devuelve un solo objeto con la farmacia más cercana según el algoritmo del profesor.

### 5.2.1. Hallazgo Técnico y Decisión de UI (Fase 4)

Mediante pruebas de integración (TDD) se comprobó que el backend del profesor **sí implementa un filtro de distancia geoespacial** (arroja 404 Not Found si se consulta desde zonas lejanas como Arica o Punta Arenas). Sin embargo, al ser un entorno de pruebas, su base de datos es sumamente reducida y suele retornar la misma farmacia ("SALVADOR GUTIERREZ") de manera constante para cualquier ubicación dentro del radio de Santiago. Por este motivo y para evitar confusiones de usabilidad en la presentación (donde el usuario vería la farmacia UTEM destacada muy lejos y las del MINSAL cerca), se tomó la decisión técnica de **desactivar temporalmente el renderizado de la capa de marcadores del MINSAL**, dejando el mapa limpio y a la espera de implementar la lógica visual definitiva en la Fase 5.

```

{
  "apertura_normal": "00:00:00",
  "cadena": "Guti",
  "cierre_normal": "00:00:00",
  "direccion": "CALLE SALVADOR GUITIERREZ 7902, TURNOS, CERRO NAVIA",
  "latitude": -33.4173332,
  "longitude": -70.7475457,
  "nombre": "SALVADOR GUTIERREZ",
  "telefono": 0,
  "tienda": 10245
}

```

**Campos rescatados:** Se rescata `nombre`, `direccion`, `latitude`, `longitude`, `apertura_normal` y `cierre_normal`.